

Space Avionics Assignment 2

LEON3 Software Tools

Module Instructor: Dr C. P. Bridges 2025/2026

Submitted by:

Student URN: 6943841

ACKNOWLEDGEMENT

I would like to thank Prof. Dr. Chris Bridges, my Space Avionics module instructor, for giving me the opportunity to work on this report. Honestly, I learned a whole new concept, both during the lab sessions and while working independently, either on the code or report. During the module lab sessions, Dr. Chris helped to clear my doubts, pushed me to use more of my brain and made sure that when I walk out of that lecture I have something new to research about and reflect on. The topic of this report has made me interested in learning more about it. I have tried to fit my learnings about this new topic in this report, and I hope to increase my knowledge in this field.

Abstract

This report investigates the LEON3 software development ecosystem, including the cross-compilation toolchain and TSIM simulation environment, as a precursor to deployment on a flight software on radiation-tolerant embedded hardware. To the address the challenges such as single-event upset (SEUs), fault-tolerant processor architecture LEON3 SPARC V8, Gaisler BCC2 cross-compiler and TSIM3 simulator by the European Space Agency (ESA) and Gaisler Research have been surveyed, followed by a practical guide to cross-compilation and execution. GitLab version control and CI/CD pipelines are evaluated in the context of the assignment. Dhrystone benchmark profiling is performed across multiple configurations such as core count, clock frequency, and hardware multiply/divide. A Hamming(31,26) code has been developed and optimised to decode the provided JPEG image targeting the LEON3 SPARC V8 architecture, starting from loop-based baseline and finalizing at lookup-table based implementation. Also, as part of the assignment, single-bit error has also been tracked and counted across the full payload decode.

OVERVIEW OF GAISLER TOOLS

A. Compiler – BCC2

The Gaisler BCC2 is an enhanced version of BCC that supports SPARC V8 architecture that supports LEON3 processors. It is based on GNU cross-compiler tools that supports C language and is used in embedded environments where code is executed directly on target hardware. BCC consists of the following packages: [1]

- GNU GCC 13.2.1 C/C++ compiler with support for atomic operations *sparc-gaisler-elf-gcc* – a C/C++ compiler front end
- *libbcc* - A user library for programming LEON systems
- *sparc-gaisler-elf-ld* – the Linker, supplied with LEON-specific memory-map linker scripts
- GNU GDB 14.2 source-level debugger
- *sparc-gaisler-elf-gdb* – a cross GDB that gets added to TSIM's server for source level debugging

B. Simulator – TSIM3 LEON3

The TSIM simulator developed by Gaisler is a generic SPARC architecture simulator capable of emulating ERC32- and LEON-based computer systems.[tsim2 manual] It allows software to be executed and analysed in a controlled environment by emulating the processor core, memory hierarchy, and peripheral interfaces. TSIM provides several unique features [2]:

- Processor Emulation (LEON3)
- Standalone + GDB Debugging
- Non-intrusive Execution Time Profiling
- Multi-core support (LEON3/4)

C. Development Workflow

The workflow in general may consist of writing C code, compiling it using SPARC GCC toolchain, linking to produce an executable, and executing the program within the TSIM simulator.

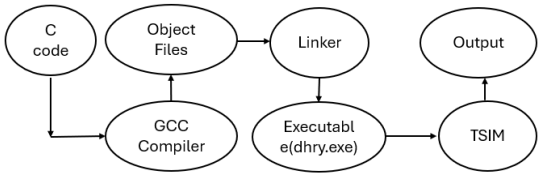


Fig 2: Workflow

D. Toolchain Limitations and Considerations [2]

The toolchain can provide a powerful development environment, but it has its own cons and limitations which must be considered:

- Simulation vs Hardware: TSIM is a simulator and can provide accurate behaviour, but it cannot fully provide results of real hardware timing.
- Cross-Compilation Complexity: Debugging can be complex due to differences between host and target architecture.
- Performance Overheads: For large workloads, simulation can be slower than real hardware.

Even after these limitations, the toolchain can be a robust and flexible platform for developing and validating embedded software prior to deployment on flight systems.

COMPILATION AND EXECUTION OF EXAMPLE CODE

This section will be demonstrating the process of compiling and executing a C program for the LEON3 processor by using the provided toolchain and simulator. The objective of this section is to confirm the development workflow and correct execution withing the TSIM environment.

A. Compilation Process:

The Dhrystone benchmark was compiled using *"sparc-elf-gcc"* SPARC cross-compiler. The executable file, *"dhry.exe"* was generated by using the "make" command and a minor warning related to the *strcpy* function was observed without affecting the successful compilation.

From the log:

```
$ make dhry.exe
sparc-elf-gcc -O2 -o dhry.exe dhry.c
In file included from dhry.c:2:
dhry_1.c:101: warning: incompatible implicit declaration of built-in function 'strcpy'
$ chmod +x dhry.exe
```

B. Execution Output:

After the compilation, the next process was executing the file using TSIM LEON3 simulator. The configuration "20 MHz core 1" is taken as example to show that the simulator successfully executed the Dhrystone benchmark and reported performance results such as execution time, Dhrystones per second, and Dhrystone MIPS. The obtained performance of 15.5 DMIPS shows the computational capability of the simulated LEON3 processor under the given configuration and the program is seen to be terminating normally confirming successful execution.

From the log:

```
Running 20MHz, 1 core, MUL/DIV ON
TSIM3 LEON3 SPARC simulator
Number of CPUs: 1
system frequency: 20.000 MHz
Execution starts, 400000 runs through Dhrystone
Microseconds for one run through Dhrystone:   36.8
Dhrystones per Second:                27173.9
Dhrystones MIPS      :                15.5
Program exited normally on CPU 0.
```

C. Execution Analysis and Discussion

The output provides information about the processor configuration, memory allocation and program execution status and the successful execution confirms correct cross-compilation, proper linking of executable and functional simulation environment.

However, it is important to keep in mind that the simulation cannot perfectly clone real hardware behaviour and final verification on hardware remains essential.

VERSION CONTROL AND CI/CD WITH GITLAB

Version Control using GitLab was implemented to manage the source code and track development progress. Git enables incremental commits, rollback and structured development. The Git workflow includes committing changes with descriptive messages, pushing updates to remote repository and maintaining version history.

A. Version Control Principles

Git is a distributed version control system that keeps the track of changes in a project through commits, each change can be identified by using SHA-1 hashes. For embedded projects like this one, branching gives freedom to change the code without getting worried about affecting the stable main branch. It is user's choice to use commits to describe their changes to keep the track of improvements and for future reference and is essential in domains like space software which requires clear tracking of changes.

B. GitLab CI/CD Pipeline

The .gitlab-ci.yml defines six stages pipeline:

stages:	
- build_single	- Compiles singles encode/decode tests
- run_single	- run under tsim-leon3
- build_unit	- compile full unit test suite
- run_unit	- fail if "Fail" found in log
- build_decode	- compile payload (JPEG) decode binary
- run_decode	- run decode, collect image + log

Table 1 : CI/CD stage definitions from gitlab-ci.yml

Each stage depends on the previous one, that means if one compilation fails, the following stages will not run. For the matter of debugging, artefacts such as binaries and logs are saved even when the job fails.

C. Continuous Integration Benefits

The pipeline is activated automatically with every "commit and push" that has the following advantages for this assignment:

- Regression detection – a bad commit breaks the unit and is caught quickly during unit testing instead of later at submission.
- Performance logging – The throughput and bit error count can be tracked over the time with each modification to enhance the performance.
- Submission confidence: a fully green pipeline demonstrates that code has compiled and executed successfully on the reference platform giving the desired output.

PERFORMANCE ANALYSIS USING DHRYSTONE BENCHMARK

[*Refer* Appendix A]

As per the requirement of the assignment the Dhrystone Benchmark [3] must be executed under multiple processor configurations of FPGA speeds of 20 MHz and 220 MHz at both core 1 and core 2, with and without multiple/divide hardware functionality and to evaluate the impact of clock frequency, number of cores, and availability of hardware arithmetic units on system performance. The results from the TSIM LEON3 simulator are analysed below.

A. Effect of Clock Frequency:

The processor is seen to be achieving a performance of 15.5 DMIPS at a clock frequency of 20 MHz but when the frequency was increased to 220 MHz, the simulator reported that the measured execution was too small to obtain meaningful results.

Output A:	Output B:
Running 20MHz, 1 core, MUL/DIV ON Number of CPUs: 1 system frequency: 20.000 MHz Execution starts, 400000 runs through Dhrystone Microseconds for one run through Dhrystone: 36.8 Dhrystones per Second: 27173.9 Dhrystones MIPS : 15.5 Program exited normally on CPU 0.	Running 220MHz, 1 core, MUL/DIV ON Number of CPUs: 1 system frequency: 220.000 MHz Execution starts, 400000 runs through Dhrystone Measured time too small to obtain meaningful results Please increased number of runs Program exited normally on CPU 0.

Table 2: Output showing Clock Frequency

This shows that the benchmark completed too quickly at higher frequencies which made it difficult to accurately measure the time of execution.

B. Effect of Number of Cores:

The results indicates that increasing the number of cores from 1 core to 2 did not result in any performance improvement. In case of 20 MHz and 220 MHz, both at core 1 and 2, the Dhrystone MIPS remained constant at 15.5 DMIPS. This is because the Dhrystone benchmark is a single-threaded workload and does not utilise multiple cores [3]. Hence, the results did not change because the additional cores were underutilised that lead to no observable performance gain.

Output A:	Output B:
Running 20MHz, 1 core, MUL/DIV ON Number of CPUs: 1 system frequency: 20.000 MHz Execution starts, 400000 runs through Dhrystone Microseconds for one run through Dhrystone: 36.8 Dhrystones per Second: 27173.9 Dhrystones MIPS : 15.5 Program exited normally on CPU 0.	**Running 220MHz, 1 core, MUL/DIV ON** Number of CPUs: 1 system frequency: 220.000 MHz Execution starts, 400000 runs through Dhrystone Measured time too small to obtain meaningful results Please increased number of runs Program exited normally on CPU 0.
Option C:	Option D:
Running 20MHz, 2 core, MUL/DIV ON Number of CPUs: 2 system frequency: 20.000 MHz	**Running 20MHz, 2 core, MUL/DIV ON** Number of CPUs: 2 system frequency: 220.000 MHz

Execution starts, 400000 runs through Dhrystone Microseconds for one run through Dhrystone: 36.8 Dhrystones per Second: 27173.9 Dhrystones MIPS : 15.5 Program exited normally on CPU 0.	Execution starts, 400000 runs through Dhrystone Measured time too small to obtain meaningful results Please increase number of runs Program exited normally on CPU 0.

Table 3: Output showing effect of number of cores

C. Effect of MUL/DIV Instructions

When the Multiply/Divide hardware functionality was disabled, the performance was again not affected as the system was still achieving 15.5 DMIPS. Hence, this suggest that the Dhrystone benchmark does not rely heavily on multiplication and division operations, or that such operations are not performance critical in this workload.

Output D:
Running NO MUL/DIV Number of CPUs: 1 system frequency: 20.000 MHz Execution starts, 400000 runs through Dhrystone Microseconds for one run through Dhrystone: 36.8 Dhrystones per Second: 27173.9 Dhrystones MIPS : 15.5 Program exited normally on CPU 0.

Table 4: Output showing effect of hardware functionality

D. Summary of the Results:

Frequency	**Cores**	**MUL/DIV**	**DMIPS**
20 MHz	1	ON	15.5
220 MHz	1	ON	N/A
20 MHz	2	ON	15.5
220 MHz	2	ON	N/A
20 MHz	1	OFF	15.5

Table 5: Output Summary

E. Tool Usage and Practical Insights [4]

Results obtained from the TSIM LEON3 simulator shows both the strengths and limitations of simulation-based performance evaluation. Even though the TSIM provides a flexible platform for testing different processor configuration without requiring physical hardware, the results also demonstrates that at higher frequencies the, the benchmark execution time became too small to be measured accurately.

Furthermore, the lack of performance variation across multi-core configurations emphasizes that the benchmarking tools must match the workload carefully. As in this case, the Dhrystone benchmark does not exploit parallelism which made it unsuitable for evaluating multi-core scalability. The automated testing across multiple configurations can be considered useful in aerospace and embedded systems, where repeatability, reliability and traceability is critical for correct results, hence careful interpretation of the results is essential to avoid misleading conclusions.

HAMMING(31, 26) ENCODER AND DECODER [5]

A. Theoretical Background

A Hamming(31,26) code adds 5 parity bits to 26 data bits which now becomes a 31-bit codeword to allow single bit error correction (SEC). The parity bits are placed at position that are power of 2, such that Parity bits = 2^n, where n = whole numbers (0, 1, 2, 3, 4…). Hence, the parity bits are at 1, 2, 4, 8, 16,… and so on namely P1, P2, P4, P8, and P16 since we are taking 31 bits into an account. Each parity bit checks a specific group of bits based on their binary positions.

Similarly, at the decoder, these parity checks are repeated, and the results form a binary value called syndrome, which is used in error correcting codes and indicates the position of any error. If the syndrome is zero, no error is present.

B. Baseline Implementation

Refer Hamming(31, 26) Encoder and Decoder C(i)

C. Optimized Implementation

(i) Design Rationale

Two key optimizations were made to improve the performance:

a. Loop Elimination: The baseline code included loops and although it was generating the result, it was still lacking in performance. Hence, to increase the performance of the code, loops were removed and the placement of 26 data bits into 31-bit codeword was implemented using direct shift (<<, >>)-and-OR operations. This removed the loop overhead and allowed the compiler to optimize instruction efficiently on the SPARC processor.

b. Lookup table (LUT) parity computation: The performance of the code was increased immensely from encoding speed 17,000 to 130,000 and decoding speed from 9,000 to 126,000 words per second once the parity calculation was improved. This speed was achieved over the course of multiple attempts of code optimization with the help of a precomputed lookup table (parity8[256]=) that stored the parity of all 8 bits values. This assisted in reducing parity computation for a 32-bit word to a constant time operation, replacing multiple iteratives XOR operations and using a few bitwise operations.

```
static const unsigned char parity8[256] = {
0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,……..//256 entries
};
```

The parity32_lut() function reduces a 32-bit word into a single parity bit using XOR explained with an example:

```
static inline unsigned int parity32_lut(unsigned int x) //unsigned bits=new bits are filled with 0s
{
   //x = 0xABCD1234 => x=10101011 11001101 00010010 00110100
   x ^= x >> 16; //x=10101011 11001101 10111001 11111001
   x ^= x >> 8; // x=10101011 01100110 01110100 01000000
   return parity8[x & 0xFF]; //Take last8 bits only
}
```

(ii) Bit error count

A global integer total_err_count is declared to track the number of corrected bit errors and is added inside hammingDecode function whenever a non-zero syndrome is detected which indicates that an error has been found and corrected. This approach introduces minimal overhead having a simple conditional check and an increment operation.

[Refer Payload Test Demonstration and Performance Improvement C (iv)*]*

(iii)Filling Data Bits

The 26-bit input data word, d, is mapped into 31-bit encoded word, e. Each data bit extract using bit shifting and masking and placed into its correct position in the encoded word using left shift. Parity bit positions, 1, 2, 4, 8 ,16 (index form: 0, 1, 3, 7, 15) are skipped for later insertion.

```
// Filling in data bits and leaving space for parity bits
//it is to be noted that space for parity bits is empyt for now which will be filled by the masking step e |= parity32_lut(e & mask) << position;
   e  = ((d >>  0) & 1u) <<  2; // shift right by 0 pos-index, do the AND operatioan and shift left 2 and store it in e, now e[2] = d[0]
```

(iv)Encoder Parity Computation

After scattering all the 26 data bits in 31-bit codeword, the 5 Parity bits are computed and inserted into 5 branch free statement. Each mask , for ex0 0x55555554, selects only the bits that belong to a specific parity group, and these groups corresponds to the Hamming rule of parity bit covering all positions where a specific bit in the index is set.

```
 //computing parity bits and insert into encoded word
   e |= parity32_lut(e & 0x55555554u) << 0;   // P1 covers bits at 1,3,5,7.... and in bit-index 0,2,4,6,8....., that is, where bit0 of bit index is equal to 1
   e |= parity32_lut(e & 0x66666666u) << 1;   // P2 covers bits at 2,3,6,7,10.... bit-index = 1,2,5,6,9..... where bit1 of bit-index is equal to 1
```

(v) Decoder syndrome computation

At the decoding end, the same parity masks are reused to recompute parity across the received word and the results are combined into a syndrome (check). e ^=(1u << (check – 1u)); flips the correct bit and restores the original codeword.

```
//Compute check via the same masks as the enoder
   check  = parity32_lut(e & 0x55555555u) << 0; //Put P1 result in bit 0
 if (check) { //if check = 0, no error: if check != 0 then error at bit0
   e ^= (1u << (check - 1u));
total_err_count++;
}
```

(vi) Filling Decoded Bits

Lastly, if there is any error and once it is correct, the original 26 data bits are extracted. The parity bit locations are skipped, ensuring only data bits are reconstructed.

```
//doing the opposite for decoding what we did in encoding step e[2] = d[0], the parity bits are getting skipped
   d  = ((e >>  2) & 1u) <<  0;
   d |= ((e >>  4) & 1u) <<
```

PAYLOAD TEST DEMONSTRATION AND PERFORMANCE IMPROVEMENT:

A. JPEG Payload Description

A JPEG image was given as payload, encoded using Hamming(31,26) error correction code prior to transmission. This encoding process introduces redundancy, that requires correction of single-bit error caused by events such as radiation induced single-event upsets (SEUs) that may occur during transmissions.

To reconstruct this image, a decoding process needs to be carried out at the receiver's end that operates sequentially on each 31-bit codeword. For every received word, the hamingDeocde function is called to compute the syndrome, detect if there is any single bit error, and apply correction as per the requirement. After the correction, the corrected data bits are then extracted and reassembled into the original byte structure, reconstructing the JPEG image progressively.

Once the 31-bit codeword is processed, the total_err_count contains the total number of detected and corrected errors single bit errors: Total bit errors detected and corrected: 393

The successful reconstruction of the JPEG Image serves as validation of the decoding process running correctly. If there is a presence of multiple bit errors corrupting the JPEG payload image, then it cannot be corrected with the application of single-error correction method.

B. Bit Error Count and Recovered Image

Payloads decode log, run_decode, gives the total corrected bit errors:

```
Number of CPUs: 2
system frequency: 50.000 MHz
Hamming decoding of payload data test:
Decoding payload data [========================================] 100.00%
Total bit errors detected and corrected: 393
Writing decoded file [========================================] 100.00%
Finished decoding payload data.
Program exited normally on CPU 0
```

The recovered JPEG payload image after Hamming(3, 26) decoding and single-bit error correction :



Fig. 1 JPEG Payload image

C. Performance Improvement Strategy

The task to decode the JPEG payload image started with a simple code whose sole purpose was to reconstruct the image and once the image was retrieved the performance optimization was carried out over the course of multiple trial with a strong focus on improving the computational efficiency and maintain correctness. As it is quite difficult to include the results and bottlenecks of each attempt, the four considerably important attempts have been provided below showing some of the major changes in the process.

(i) Baseline Implementation: Array + Loops [*Refer* Appendix B]

- The initial implementation used a very straightforward approach based on array manipulation. For the encoding process, the input word was first unpacked into an array of individual bits using unpackbits() and data bits were placed into appropriate positions, while the parity bits were computed using nested loops that iterated over all 31-bit positions for each parity group.
- For the decoding process, array-based operations were used where the syndrome was calculated using multiple nested loops.
- As mentioned previously as well, while the implementation was functionally correct and helped to lay the foundation of understand encoding-decoding procedure, but it introduced significant computational overhead because of repeated use of loops, conditional checks affecting the critical factors of LEON3 processors.
- The measured performance for this version was:
  - Encoding speed: 5329 words per second
  - Decoding speed: 5448 words per second

(ii) Bitwise Optimization [*Refer* Appendix C]

- Once the image was retrieved, now it was time to move on to process of optimizing the code to make the run efficient. The array-based operations were removed and replaced direct bitwise manipulation. Instead of unpacking data into arrays, each bit were accessed using shift and mask operations that allowed the encoded word to be constructed within a 32-bit integer.
- In addition to that, a fast parity computation function (partiy32) was introduce using XOR folding techniques that reduced the cost of parity calculations compared to iterative methods.
- Even though there was a noticeable increase in the encoding speed because of implementation of bitwise operation, the decoding process was still using loops for parity computation and syndrome calculation. Hence, the decode was still not showing a significant improve. So, the next step was to replace loops entirely from the code.
- The measured performance for version was:
  - Encoding speed: 17,991 words per second
  - Decoding speed: 7,484 words per second

(iii) Loop Elimination and LUT-based parity I[*Refer* Appendix D]

- In this version, the loops were removed entirely from the code, and parity computation was replaced with lookup table (LUT)-based method. A precomputed

table (parity8[256]) stored the parity of all possible 8-bit values reducing the complexity of parity computation.

- It is to be noted that till now there was no mention of total error count and correction. So along with performance optimization, this version tries to count and print the number of single-bit errors.
- Even though the encoding performance showed a massive improvement, the decoding performance got extremely slow because of debugging statement, printf, was left inside the decoding function and executed for every processed word.
- The measured performance for this version was:
  - Encoding speed: 130, 284 words per second
  - Decoding speed: 6,509 words per second

(iv) Final Optimize Implementation [*Refer* Appendix E]

- After multiple attempts to count and print the single-bit errors, the debugging output was removed from the decoding path.
- The optimization needed to be made inside functions decode_payload.c and hamming.h along with hammin.c to print the out of error count.
  - hamming.c :

```
if (check) { //if check = 0, no error: if check != 0 then error at bit0
e ^= (1u << (check - 1u));
total_err_count++;
}
```

  - decode_payload.c :

```
printf("\n\nTotal bit errors detected and corrected: %d\n", total_err_count);  // to count the total number of errors
```

  - hamming.h :

```
extern int total_err_count;
```

- Both encoding and decoding were optimized with bitwise operations and LUT-based parity computation and bit masks were used efficiently to isolate relevant bits for each parity group.
- The measured performance was this final version was:
  - Encoding speed: 130,281 words per second
  - Decoding speed: 126,319 words per second

References

[1] F. Gaisler, “BCC User's Manual,” 2024.

[2] COBHAM, “TSIM2 Simulator User's Manual,” 2020.

[3] NXP, “NXP,” NXP Semiconductors, 23 January 2025. [Online]. Available: https://docs.nxp.com/bundle/AN13917/page/topics/dhrystone.html.

[4] R. P. Weicker, “Dhrystone: a synthetic systems programming benchmark,” Association for Computing Machinery, 1984.

[5] W. c. “Hamming code,” Wikipedia, The Free Encyclopedia., April 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Hamming_code&oldid=1351548572.

Appendix A

```
#dhry
build_dhry: #defines a job
  stage: build #the build stage runs before any "run" jobs
  script:
    - cd $CI_PROJECT_DIR #gitlab working directory
    - make dhry.exe | tee build_dhry_log.txt #builds excutable | saves output and prints
    - chmod +x dhry.exe #makes executable runnable
  artifacts: #saves files after job finishes
    paths:
     - dhry.exe
     - build_dhry_log.txt #for debugging

#case 1: 20 MHz, 1 core
run_20_1:
  stage: run
  dependencies:
    - build_dhry #uses output from build_dhry
  script:
    - cd $CI_PROJECT_DIR
    - echo "Running 20MHz, 1 core, MUL/DIV ON" #prints messages/comments in logs
    - tsim-leon3 -freq 20 -numcpus 1 -nov8 0 dhry.exe -c tsim_commands | tee dhry_20MHz_1core_mul.txt # -
frq 20 = sets CPU frequency = 20 MHz, -numcpus 1 = defines the processor, -nov8 0 = enables MUL/DIV |
0=enabled, 1=disabled, dhry.exe = program excution, -c tsim_commands = loads simulator configuration file |
tee dhry_20MHz_1core_mul.txt = saves output to file
  #TSIM does not support changing hardware via .cmd files
  artifacts: #saves result file
    when: always
    paths:
     - dhry_20MHz_1core_mul.txt

#case 2: 220 MHz, 1 core
run_220_1:
  stage: run
  dependencies:
    - build_dhry
  script:
    - cd $CI_PROJECT_DIR
    - echo "Running 220MHz, 1 core, MUL/DIV ON"
    - tsim-leon3 -freq 220 -numcpus 1 -nov8 0 dhry.exe -c tsim_commands | tee dhry_220MHz_1core_mul.txt
  artifacts:
    when: always
    paths:
     - dhry_220MHz_1core_mul.txt
#case 3: 20 MHz, 2 core
run_20_2:
  stage: run
  dependencies:
    - build_dhry
  script:
    - cd $CI_PROJECT_DIR
    - echo "Running 20MHz, 2 core, MUL/DIV ON "
    - tsim-leon3 -freq 20 -numcpus 2 -nov8 0 dhry.exe -c tsim_commands | tee dhry_20MHz_2core_mul.txt
```

```
  artifacts:
    when: always
    paths:
     - dhry_20MHz_2core_mul.txt
#case 4: 220 MHz, 2 core
run_220_2:
  stage: run
  dependencies:
    - build_dhry
  script:
    - cd $CI_PROJECT_DIR
    - echo "Running 220MHz, 2 cores, MUL/DIV ON"
    - tsim-leon3 -freq 220 -numcpus 2 -nov8 0 dhry.exe -c tsim_commands | tee dhry_220MHz_2core_mul.txt
  artifacts:
    when: always
    paths:
     - dhry_220MHz_2core_mul.txt
#case 5: No MUL/DIV
run_no_mul:
  stage: run
  dependencies:
    - build_dhry
  script:
    - cd $CI_PROJECT_DIR
    - echo "Running NO MUL/DIV"
    - tsim-leon3 -freq 20 -numcpus 1 -nov8 1 dhry.exe -c tsim_commands | tee dhry_nomul.txt
  artifacts:
    when: always
    paths:
     - dhry_nomul.txt
```

Appendix B

Version 1 – Baseline (Array + Loops)

```
#include "hamming.h"
int total_error_count = 0;
void unpackBits(HC_WORD inputWord, int bitCount, unsigned int *outputArray)
{
  int bit;
  for (bit=0; bit<bitCount; ++bit)
    outputArray[bit] = (inputWord >> bit) & 0x01;
}
HC_WORD packBits(unsigned int *inputArray, int bitCount)
{
  int bit;
  unsigned int outputWord = 0;
  for (bit=0; bit<bitCount; ++bit)
    outputWord |= (inputArray[bit] & 0x01) << bit;
  return outputWord;
}
void printWordDebug(unsigned int word)
{
  int i;
```

```
  printf("lsb [");
  for (i=0; i<32; ++i)
  {
    if ((word >> i) & 0x1 == 1)
            printf("1");
    else
            printf("0");
  }
  printf("] msb");
}
HC_WORD hammingEncode(HC_WORD rawData)
{
  HC_WORD encodedData;
  unsigned int input_bits[DECODED_SIZE];
  unsigned int output_bits[ENCODED_SIZE];
  encodedData = packBits(output_bits, ENCODED_SIZE);
int i, j;
j = 0;
for (i = 0; i < ENCODED_SIZE; i++) {
    if (i == 0 || i == 1 || i == 3 || i == 7 || i == 15) {
        output_bits[i] = 0;
    } else {
        output_bits[i] = input_bits[j++];
    }
}
int p;
for (p = 0; p < 5; p++) {
    int parity = 0;

    for (i = 0; i < ENCODED_SIZE; i++) {
        if ((i + 1) & (1 << p)) {
            parity ^= output_bits[i];
        }
    }

    output_bits[(1 << p) - 1] = parity;
}
encodedData = packBits(output_bits, ENCODED_SIZE);
return encodedData;
}

HC_WORD hammingDecode(HC_WORD encodedData)
{
  HC_WORD decodedData;
  unsigned int input_bits[ENCODED_SIZE];
  unsigned int output_bits[DECODED_SIZE];
  unpackBits(encodedData, ENCODED_SIZE, input_bits);
  decodedData = packBits(output_bits, DECODED_SIZE);
int i, j;
int error_pos = 0;
static int printed = 0;
/* Step 1: syndrome */
int p;
for (p = 0; p < 5; p++) {
    int parity = 0;
    for (i = 0; i < ENCODED_SIZE; i++) {
```

```
        if ((i + 1) & (1 << p)) {
            parity ^= input_bits[i];
        }
    }
    if (parity != 0) {
        error_pos += (1 << p);
    }
}
if (error_pos != 0) {
    input_bits[error_pos - 1] ^= 1;
    total_error_count++;
}
j = 0;
for (i = 0; i < ENCODED_SIZE; i++) {
    if (!(i == 0 || i == 1 || i == 3 || i == 7 || i == 15)) {
        output_bits[j++] = input_bits[i];
    }
}
decodedData = packBits(output_bits, DECODED_SIZE);
if (!printed && total_error_count > 0) {
    printf("Total bit errors corrected: %d\n", total_error_count);
    printed = 1;
}
return decodedData;
}
```

Appendix C

Version 2 – Bitwise Optimization

```
#include "hamming.h"
static inline int parity32(unsigned int x)
{
    x ^= x >> 16;
    x ^= x >> 8;
    x ^= x >> 4;
    x &= 0xF;
    return (0x6996 >> x) & 1;
}
int total_error_count = 0;
void unpackBits(HC_WORD inputWord, int bitCount, unsigned int *outputArray)
{
  int bit;
  for (bit=0; bit<bitCount; ++bit)
    outputArray[bit] = (inputWord >> bit) & 0x01;
}
HC_WORD packBits(unsigned int *inputArray, int bitCount)
{
  int bit;
  unsigned int outputWord = 0;
  for (bit=0; bit<bitCount; ++bit)
    outputWord |= (inputArray[bit] & 0x01) << bit;
  return outputWord;
}
void printWordDebug(unsigned int word)
```

```
{
  int i;

  printf("lsb [");
  for (i=0; i<32; ++i)
  {
    if ((word >> i) & 0x1 == 1)
            printf("1");
    else
            printf("0");
  }
  printf("] msb");
}
HC_WORD hammingEncode(HC_WORD d)
{
    unsigned int e = 0;
    e |= ((d >> 0) & 1) << 2;
    e |= ((d >> 1) & 1) << 4;
    e |= ((d >> 2) & 1) << 5;
    e |= ((d >> 3) & 1) << 6;
    e |= ((d >> 4) & 1) << 8;
    e |= ((d >> 5) & 1) << 9;
    e |= ((d >> 6) & 1) << 10;
    e |= ((d >> 7) & 1) << 11;
    e |= ((d >> 8) & 1) << 12;
    e |= ((d >> 9) & 1) << 13;
    e |= ((d >> 10) & 1) << 14;
    e |= ((d >> 11) & 1) << 16;
    e |= ((d >> 12) & 1) << 17;
    e |= ((d >> 13) & 1) << 18;
    e |= ((d >> 14) & 1) << 19;
    e |= ((d >> 15) & 1) << 20;
    e |= ((d >> 16) & 1) << 21;
    e |= ((d >> 17) & 1) << 22;
    e |= ((d >> 18) & 1) << 23;
    e |= ((d >> 19) & 1) << 24;
    e |= ((d >> 20) & 1) << 25;
    e |= ((d >> 21) & 1) << 26;
    e |= ((d >> 22) & 1) << 27;
    e |= ((d >> 23) & 1) << 28;
    e |= ((d >> 24) & 1) << 29;
    e |= ((d >> 25) & 1) << 30;

    int i;
    int p1 = 0, p2 = 0, p4 = 0, p8 = 0, p16 = 0;

    for (i = 1; i <= 31; i++) {
        int bit = (e >> (i - 1)) & 1;

        if (i & 1)  p1 ^= bit;
        if (i & 2)  p2 ^= bit;
        if (i & 4)  p4 ^= bit;
        if (i & 8)  p8 ^= bit;
        if (i & 16) p16 ^= bit;
    }
```

```
    e |= (p1 << 0);
    e |= (p2 << 1);
    e |= (p4 << 3);
    e |= (p8 << 7);
    e |= (p16 << 15);

    return e;
}
HC_WORD hammingDecode(HC_WORD e)
{
    int i, p;
    int error_pos = 0;

    // -------- SYNDROME --------
    for (p = 0; p < 5; p++) {
        int parity = 0;

        for (i = 1; i <= 31; i++) {
            if (i & (1 << p)) {
                parity ^= (e >> (i - 1)) & 1;
            }
        }

        if (parity)
            error_pos |= (1 << p);
    }
    if (error_pos != 0) {
        e ^= (1 << (error_pos - 1));
        total_error_count++;
    }
    unsigned int d = 0;
    int data_pos = 0;

    for (i = 1; i <= 31; i++) {
        if (i == 1 || i == 2 || i == 4 || i == 8 || i == 16)
            continue;

        if ((e >> (i - 1)) & 1)
            d |= (1 << data_pos);

        data_pos++;
    }
    return d;
}
```

Appendix D

Version 3 – Loop Elimination and LUT-Based Parity

```
#include "hamming.h"
  int total_error_count = 0;
static const unsigned char parity8[256] = {
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
```

```
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0
};
parity32_lut(unsigned int x)
{
    x ^= x >> 16;
    x ^= x >> 8;
    return parity8[x & 0xFF];
}
#define MASK_P1  0x56AAAAABUL   /* bits at Hamming positions 1,3,5,...31  */
#define MASK_P2  0x66666666UL   /* bits at Hamming positions 2,3,6,7,...  */
#define MASK_P4  0x07878787UL   /* bits at Hamming positions 4-7,12-15,.. */
#define MASK_P8  0x0007F80FUL   /* bits at Hamming positions 8-15,24-31   */
#define MASK_P16 0x7FFF8000UL   /* bits at Hamming positions 16-31        */
void unpackBits(HC_WORD inputWord, int bitCount, unsigned int *outputArray)
{
    int bit;
    for (bit = 0; bit < bitCount; ++bit)
        outputArray[bit] = (inputWord >> bit) & 0x01;
}
HC_WORD packBits(unsigned int *inputArray, int bitCount)
{
    int bit;
    unsigned int outputWord = 0;
    for (bit = 0; bit < bitCount; ++bit)
        outputWord |= (inputArray[bit] & 0x01) << bit;
    return outputWord;
}

void printWordDebug(unsigned int word)
{
    int i;
    printf("lsb [");
    for (i = 0; i < 32; ++i)
        printf("%c", ((word >> i) & 1) ? '1' : '0');
    printf("] msb");
}
    e  = ((d >>  0) & 1u) <<  2;
    e |= ((d >>  1) & 1u) <<  4;
    e |= ((d >>  2) & 1u) <<  5;
    e |= ((d >>  3) & 1u) <<  6;
    e |= ((d >>  4) & 1u) <<  8;
    e |= ((d >>  5) & 1u) <<  9;
    e |= ((d >>  6) & 1u) << 10;
    e |= ((d >>  7) & 1u) << 11;
```

```
    e |= ((d >>  8) & 1u) << 12;
    e |= ((d >>  9) & 1u) << 13;
    e |= ((d >> 10) & 1u) << 14;
    e |= ((d >> 11) & 1u) << 16;
    e |= ((d >> 12) & 1u) << 17;
    e |= ((d >> 13) & 1u) << 18;
    e |= ((d >> 14) & 1u) << 19;
    e |= ((d >> 15) & 1u) << 20;
    e |= ((d >> 16) & 1u) << 21;
    e |= ((d >> 17) & 1u) << 22;
    e |= ((d >> 18) & 1u) << 23;
    e |= ((d >> 19) & 1u) << 24;
    e |= ((d >> 20) & 1u) << 25;
    e |= ((d >> 21) & 1u) << 26;
    e |= ((d >> 22) & 1u) << 27;
    e |= ((d >> 23) & 1u) << 28;
    e |= ((d >> 24) & 1u) << 29;
    e |= ((d >> 25) & 1u) << 30;
    e |= parity32_lut(e & 0x55555554u) << 0;   /* P1 at bit 0 */
    e |= parity32_lut(e & 0x66666666u) << 1;   /* P2 at bit 1 */
    e |= parity32_lut(e & 0x78787878u) << 3;   /* P4 at bit 3 */
    e |= parity32_lut(e & 0x7F807F80u) << 7;   /* P8 at bit 7 */
    e |= parity32_lut(e & 0x7FFF8000u) << 15;  /* P16 at bit 15 */

    return e;
}
HC_WORD hammingDecode(HC_WORD e)
{
    register unsigned int syndrome;
    register unsigned int d;
    syndrome  = parity32_lut(e & 0x55555555u) << 0;
    syndrome |= parity32_lut(e & 0x66666666u) << 1;
    syndrome |= parity32_lut(e & 0x78787878u) << 2;
    syndrome |= parity32_lut(e & 0x7F807F80u) << 3;
    syndrome |= parity32_lut(e & 0x7FFF8000u) << 4;
  if (syndrome) {
    total_error_count++;
    //printf("Bit error at position %u\n", syndrome);
    e ^= (1u << (syndrome - 1u));
}
printf("Total errors so far: %d\n", total_error_count);
    d  = ((e >>  2) & 1u) <<  0;
    d |= ((e >>  4) & 1u) <<  1;
    d |= ((e >>  5) & 1u) <<  2;
    d |= ((e >>  6) & 1u) <<  3;
    d |= ((e >>  8) & 1u) <<  4;
    d |= ((e >>  9) & 1u) <<  5;
    d |= ((e >> 10) & 1u) <<  6;
    d |= ((e >> 11) & 1u) <<  7;
    d |= ((e >> 12) & 1u) <<  8;
    d |= ((e >> 13) & 1u) <<  9;
    d |= ((e >> 14) & 1u) << 10;
    d |= ((e >> 16) & 1u) << 11;
    d |= ((e >> 17) & 1u) << 12;
    d |= ((e >> 18) & 1u) << 13;
    d |= ((e >> 19) & 1u) << 14;
```

```
    d |= ((e >> 20) & 1u) << 15;
    d |= ((e >> 21) & 1u) << 16;
    d |= ((e >> 22) & 1u) << 17;
    d |= ((e >> 23) & 1u) << 18;
    d |= ((e >> 24) & 1u) << 19;
    d |= ((e >> 25) & 1u) << 20;
    d |= ((e >> 26) & 1u) << 21;
    d |= ((e >> 27) & 1u) << 22;
    d |= ((e >> 28) & 1u) << 23;
    d |= ((e >> 29) & 1u) << 24;
    d |= ((e >> 30) & 1u) << 25;
    return d;
    //printf("Total errors so far: %d\n", total_error_count);
}
```

Appendix E

Version 4

```
#include "hamming.h"
//8bit even parity Lookup table
//precomputnig parity for all 8bit numbers so we never have to calculate it again
   int total_err_count = 0;
static const unsigned char parity8[256] = {
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1,
    0,1,1,0,1,0,0,1,1,0,0,1,0,1,1,0
};

//reduce a 32-bit word to its even parity bit using the LUT
static inline unsigned int parity32_lut(unsigned int x) //unsigned bits=new bits are filled with 0s
{
    //x = 0xABCD1234 => x=10101011 11001101 00010010 00110100
    x ^= x >> 16; //x=10101011 11001101 10111001 11111001
    x ^= x >> 8; // x=10101011 01100110 01110100 01000000
    return parity8[x & 0xFF]; //Take last8 bits only
}


//Bit filters = picks only the bits that belong to a specific parity group
//Maksing only selects the required bits for parity
// 1 hex=4 binary bits, 16=2^4, group in bits of 4 bits
```

```
// After converting to Hex, Mask becomes 0x55555555
// Now adjusting the P1, P2, P4, P8 and P16 and again converting to Hex we get 0x56AAAAAB where UL
unsigned long ensures that number is treated as a 32-bit unsigned
// now we repeat the same procedure for MASK_ P2, P4, P8, P16
#define MASK_P1  0x56AAAAABUL  /* bits at Hamming positions 1,3,5,...31  */
#define MASK_P2  0x66666666UL   /* bits at Hamming positions 2,3,6,7,...  */
#define MASK_P4  0x07878787UL   /* bits at Hamming positions 4-7,12-15,.. */
#define MASK_P8  0x0007F80FUL   /* bits at Hamming positions 8-15,24-31   */
#define MASK_P16 0x7FFF8000UL   /* bits at Hamming positions 16-31        */

// #inputWord: this takes a number and breaks it into individual bits
// bticount = bits you want to extract
// unsigned int *outputArray = array where bits will be stored
//to make it easier to work with an array of bits than the numbers
//array is used for debugging and solving the hamming
void unpackBits(HC_WORD inputWord, int bitCount, unsigned int *outputArray)
{
   int bit;
   for (bit = 0; bit < bitCount; ++bit)
      outputArray[bit] = (inputWord >> bit) & 0x01; // >> shift right by bit: & 0x01 = keeps the last bit only
}

//it takes an array of bits and builds a single integer from them
// integer is useful as a final optimized code and in embedded system
HC_WORD packBits(unsigned int *inputArray, int bitCount)
{
   int bit;
   unsigned int outputWord = 0;
   for (bit = 0; bit < bitCount; ++bit)
      outputWord |= (inputArray[bit] & 0x01) << bit;
   return outputWord;
}

//It is useful for debugging and visual verfication of bit operation, in validating Hamming encodifng and
decoding
void printWordDebug(unsigned int word)
{
   int i;
   printf("lsb [");
   for (i = 0; i < 32; ++i)
      printf("%c", ((word >> i) & 1) ? '1' : '0');
   printf("] msb");
}

HC_WORD hammingEncode(HC_WORD d) //d = 26bit data
{
   register unsigned int e; // e = 31 bit Hamming code


// Not used an loop for faster execution of the code
// Filling in data bits and leaving space for parity bits
//it is to be noted that space for parity bits is empyt for now which will be filled by the masking step e |=
parity32_lut(e & mask) << position;
   e  = ((d >>  0) & 1u) <<  2; // shift right by 0 pos-index, do the AND operatioan and shift left 2 and store it in
e, Now e[2] = d[0]
```

```
    e |= ((d >>  1) & 1u) <<  4; // shift right 1 pos-index, do AND operation with 1 and shift left by 4 pos-index
and store it in e[4]=d[1] and so on
    e |= ((d >>  2) & 1u) <<  5; // and so on
    e |= ((d >>  3) & 1u) <<  6;
    e |= ((d >>  4) & 1u) <<  8;
    e |= ((d >>  5) & 1u) <<  9;
    e |= ((d >>  6) & 1u) << 10;
    e |= ((d >>  7) & 1u) << 11;
    e |= ((d >>  8) & 1u) << 12;
    e |= ((d >>  9) & 1u) << 13;
    e |= ((d >> 10) & 1u) << 14;
    e |= ((d >> 11) & 1u) << 16;
    e |= ((d >> 12) & 1u) << 17;
    e |= ((d >> 13) & 1u) << 18;
    e |= ((d >> 14) & 1u) << 19;
    e |= ((d >> 15) & 1u) << 20;
    e |= ((d >> 16) & 1u) << 21;
    e |= ((d >> 17) & 1u) << 22;
    e |= ((d >> 18) & 1u) << 23;
    e |= ((d >> 19) & 1u) << 24;
    e |= ((d >> 20) & 1u) << 25;
    e |= ((d >> 21) & 1u) << 26;
    e |= ((d >> 22) & 1u) << 27;
    e |= ((d >> 23) & 1u) << 28;
    e |= ((d >> 24) & 1u) << 29;
    e |= ((d >> 25) & 1u) << 30;

    //computing parity bits and insert into encoded word
    e |= parity32_lut(e & 0x55555554u) << 0;   // P1 coveres bits at 1,3,5,7.... and in bit-index 0,2,4,6,8....., that
is, where bit0 of bit index is equal to 1
    e |= parity32_lut(e & 0x66666666u) << 1;   // P2 covers bits at 2,3,6,7,10.... bit-index = 1,2,5,6,9..... where
bit1 of bit-index is equal to 1
    e |= parity32_lut(e & 0x78787878u) << 3;   // P4 covers 4,5,6,7,12...bit index = 3,4,5,6,11.....where bit2 bit
index = 1
    e |= parity32_lut(e & 0x7F807F80u) << 7;   // P8 covers 8-15,24-31, bitindex 7-14,23-30, bit3 of bit index=1
    e |= parity32_lut(e & 0x7FFF8000u) << 15;  // P16 covers 16-31, bit index = 15-30, bit4 of bitindex = 1

    return e;
}


// detects error via check, fixes the rror and recovers original data, and also counts total number or errors
HC_WORD hammingDecode(HC_WORD e) //e has the 31 bit encoded word, both parity and data
{
    register unsigned int check;
    register unsigned int d;

    //Compute check via the same masks as the enoder
    check  = parity32_lut(e & 0x55555555u) << 0; //Put P1 result in bit 0
    check |= parity32_lut(e & 0x66666666u) << 1; // put P2 result in bit 1
    check |= parity32_lut(e & 0x78787878u) << 2; //Put P3 result in bit 2
    check |= parity32_lut(e & 0x7F807F80u) << 3; //Put P4 result in bit 3
    check |= parity32_lut(e & 0x7FFF8000u) << 4; //Put P5 result in bit 4

  if (check) { //if check = 0, no error: if check != 0 then error at bit0
    e ^= (1u << (check - 1u));
```

```
    total_err_count++;
}
//
    //doing the opposite for decoding what we did in encoding step e[2] = d[0], the parity bits are gettin skipped
    d  = ((e >>  2) & 1u) <<  0;
    d |= ((e >>  4) & 1u) <<  1;
    d |= ((e >>  5) & 1u) <<  2;
    d |= ((e >>  6) & 1u) <<  3;
    d |= ((e >>  8) & 1u) <<  4;
    d |= ((e >>  9) & 1u) <<  5;
    d |= ((e >> 10) & 1u) <<  6;
    d |= ((e >> 11) & 1u) <<  7;
    d |= ((e >> 12) & 1u) <<  8;
    d |= ((e >> 13) & 1u) <<  9;
    d |= ((e >> 14) & 1u) << 10;
    d |= ((e >> 16) & 1u) << 11;
    d |= ((e >> 17) & 1u) << 12;
    d |= ((e >> 18) & 1u) << 13;
    d |= ((e >> 19) & 1u) << 14;
    d |= ((e >> 20) & 1u) << 15;
    d |= ((e >> 21) & 1u) << 16;
    d |= ((e >> 22) & 1u) << 17;
    d |= ((e >> 23) & 1u) << 18;
    d |= ((e >> 24) & 1u) << 19;
    d |= ((e >> 25) & 1u) << 20;
    d |= ((e >> 26) & 1u) << 21;
    d |= ((e >> 27) & 1u) << 22;
    d |= ((e >> 28) & 1u) << 23;
    d |= ((e >> 29) & 1u) << 24;
    d |= ((e >> 30) & 1u) << 25;

    return d;
}
```

----------*END OF REPORT*----------